

Educational STC89C52 Microcontroller Simulator with Process Scheduling

Team Members

Member	Primary Responsibility	Primary Responsibility
Student 1 — Team Leader Reg no.25190152	CPU & Instruction Execution	Integration & GitHub
Student 2 Reg no.25190155	Memory & Stack	CPU support
Student 3 Reg no.25190153	Data Structures & Process Management	Testing
Student 4 Reg no.25190140	OS Scheduling & Context Switching	UI & Integration

problem Objective

Build a simplified educational simulator that demonstrates instruction execution, memory and peripheral operations, and process management on the STC89C52 ,8051 family microcontroller, integrating concepts from Computer Architecture, Data Structure, and Operating System into one working system.

Problem Statement

Design and implement a software-based simulator for the assigned 8-bit microcontroller processor. The simulator models the essential processor components, executes a defined subset of its instructions, manages program memory, data memory, and stack, and provides simplified GPIO, timer, and interrupt functionality. The simulator also supports multiple programs as processes using appropriate data structures — a Process Control Block (PCB), ready queue, context switching — and implements FCFS, Round Robin, and Priority CPU scheduling algorithms.

Project Scope

In scope

- A simplified subset of the STC89C52 (8051) instruction set (data transfer, arithmetic, logic, branch/jump, call/return, stack operations)
- Registers (A, B, R0–R7, SP, PC), a subset of the PSW flags, data memory, a call/push stack
- Simplified GPIO ports (P0–P3), a single timer (Timer0) with overflow interrupt, and one ISR
- Multiple loaded programs represented as OS-style processes (PCB, states, ready queue)
- FCFS, Round Robin, and Priority CPU scheduling with context switching
- Load / Run / Step / Reset controls and live visualization of CPU, memory, GPIO, and scheduler
- Performance metrics: waiting time, turnaround time, response time, context switches, CPU utilization

Out of scope

- Full instruction set / all addressing modes of the real STC89C52
- Real hardware timing accuracy, clock cycles, or electrical behavior
- Serial port (UART), ADC, or other advanced on-chip peripherals
- Multi-core / true concurrency (the simulator is single-CPU, time-shared)

Microcontroller Being Simulated

STC89C52 — an 8051-family, 8-bit microcontroller (8 KB Flash, 256 bytes internal RAM, 3 timers in the real chip; this project models a reduced subset: Timer0 only, as scoped above).

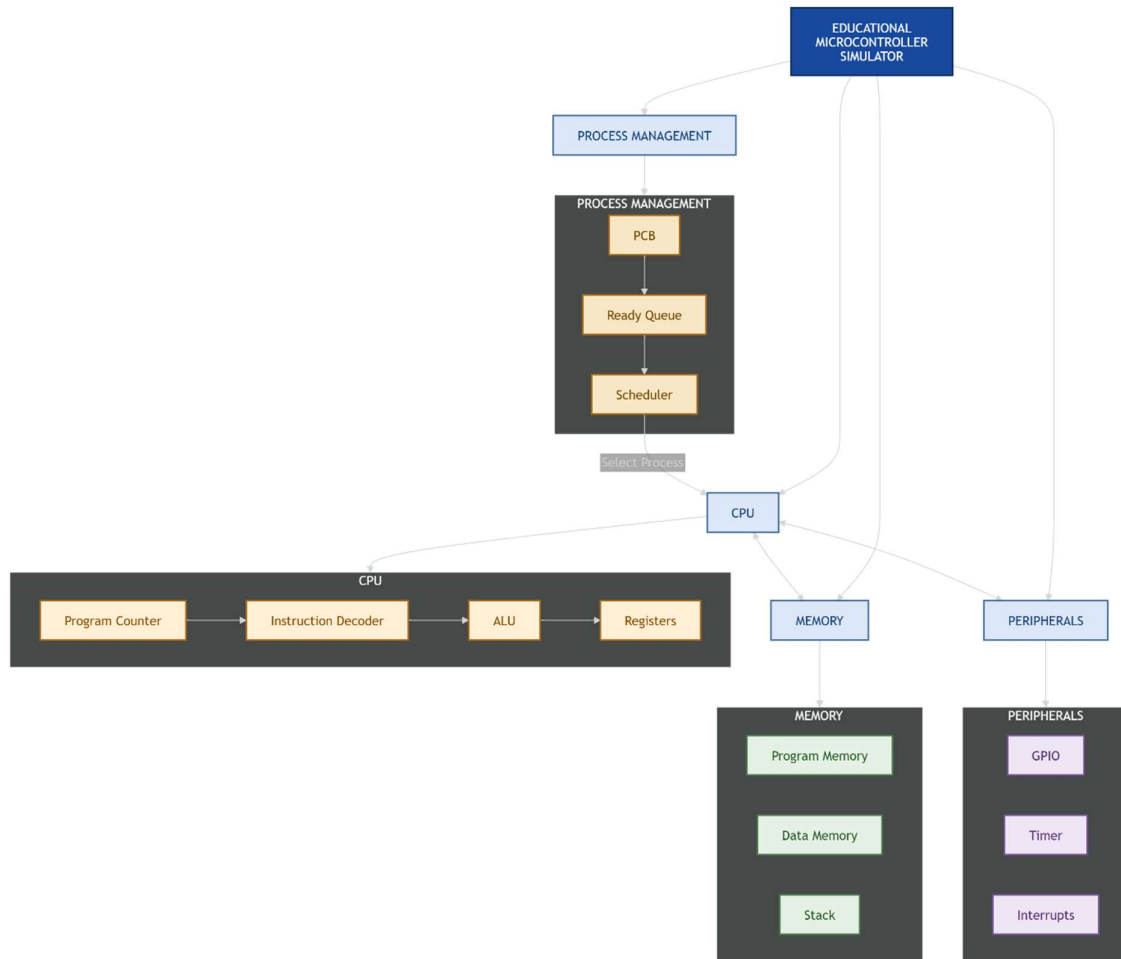
Selected Programming Language.

Java is a chosen programming language for this simulation because:

It supports object-oriented programming, which is suitable for modelling CPU, memory, registers, and process

Java provides useful data structures for implementing process scheduling, queues, stacks, and memory

Initial System Architecture



- **CPU:** registers, flags, PC, instruction execution engine, instruction lookup/dispatch table
- **Memory:** data memory (internal RAM model) + call/push stack per process
- **Peripherals:** GPIO port registers, Timer0 with overflow flag, interrupt enable bits and ISR vectoring
- **ProcessManager:** **Process Manager** manages multiple processes/programs in the simulator.
It maintains each process's **PCB, state, and ready queue** and handles context switching.
It uses scheduling algorithms like **FCFS, Round Robin, and Priority** to decide which process runs next.
- **Scheduler:** FCFS / Round Robin / Priority, ready queue, context switch bookkeeping
- **User Interface:** load/run/step/reset controls, register/memory/GPIO/timer views, ready queue view, Gantt chart, performance metrics table

Initial Development Plan

1. Team formation, repo setup, architecture study, language finalization
2. CPU core: registers, flags, memory model, instruction lookup table skeleton
3. Instruction execution engine (data transfer, arithmetic, logic, branch)
4. Stack, GPIO, Timer0, interrupt/ISR vectoring
5. PCB, process states, Queue/Circular Queue data structures
6. FCFS + Round Robin schedulers, context switching
7. Priority scheduler, performance metrics (WT/TAT/RT/CPU utilization)
8. UI: load/run/step/reset, register/memory/GPIO views
9. Ready queue view, Gantt chart, metrics dashboard
10. Integration testing, sample programs, bug fixing
11. Documentation, final report, demo prep
12. Buffer / polish / presentation